

297. Serialize and Deserialize Binary Tree **Hard**

Acceptance: 60.2% | [LeetCode](#)

Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

Clarification: The input/output format is the same as [how LeetCode serializes a binary tree](#). You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.

Example 1:

```
**Input:** root = [1,2,3,null,null,4,5]
**Output:** [1,2,3,null,null,4,5]
```

Example 2:

```
**Input:** root = []
**Output:** []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 104]`.
- `-1000 <= Node.val <= 1000`

Claude Opus Solution

Serialize and Deserialize Binary Tree

1. Intuition

The core challenge is **encoding tree structure into a flat string** and then **reconstructing it perfectly**.

Key insight: A tree's structure is fully captured if we record: - Every node's value, AND - Every `null` position (the "absence" of a node)

Without marking nulls, we couldn't distinguish between different tree shapes. For example, `[1, 2]` could mean "2 is left child" or "2 is right child" — the `null` markers disambiguate this.

I'll present a **DFS (preorder) approach** because it's cleaner than BFS — preorder naturally lets us rebuild the tree recursively in the same order we wrote it.

2. Approach

Serialize (Preorder DFS):

```
serialize(node):
    if node is null:
        append "#" to result
        return
    append node.val to result
    serialize(node.left)    # left subtree first
    serialize(node.right)   # then right subtree
```

Deserialize (Preorder DFS):

```
deserialize(tokens):
    val = next token
    if val == "#":
        return null
    node = new TreeNode(val)
    node.left = deserialize(tokens)    # consume left subtree
    node.right = deserialize(tokens)   # consume right subtree
    return node
```

The magic: deserialize consumes tokens **in the exact same order** serialize produced them. An iterator/index acts as a shared "cursor" walking through the tokens.

3. Python Solution

```
from collections import deque
from typing import Optional

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Codec:
    def serialize(self, root: Optional[TreeNode]) -> str:
        """Encode tree to a single string using preorder DFS."""
        result = [] # collect tokens here

        def dfs(node):
            # Base case: null node -> sentinel marker
            if node is None:
                result.append("#")
                return
            # Preorder: record value, then recurse left, then right
            result.append(str(node.val))
            dfs(node.left)
            dfs(node.right)

        dfs(root)
        # Join with commas (handles negative numbers & multi-digit safely)
        return ",".join(result)

    def deserialize(self, data: str) -> Optional[TreeNode]:
        """Decode string back to tree using preorder DFS."""
        # deque lets us popleft() in O(1) - acts as our token cursor
        tokens = deque(data.split(","))

        def dfs():
            # Consume the next token (same order serialize wrote them)
            val = tokens.popleft()
            # "#" means there's no node here
            if val == "#":
                return None
            # Build node, then reconstruct its subtrees IN ORDER
            node = TreeNode(int(val))
            node.left = dfs() # left subtree consumes its tokens
            node.right = dfs() # right subtree consumes the rest
            return node

        return dfs()
```

Why this works

- **Serialize** writes: root, left-subtree..., right-subtree...
- **Deserialize** reads in the same order: read root, then *fully* build left subtree (which greedily consumes exactly its tokens), then build right subtree from what remains.
- The `deque` cursor guarantees each recursive call picks up exactly where the previous left off.

4. Complexity Analysis

Operation	Time	Space
Serialize	$O(n)$	$O(n)$
Deserialize	$O(n)$	$O(n)$

Time — $O(n)$: Each node (and each null marker, of which there are at most $n+1$) is visited exactly once. String join/split are linear.

Space — O(n): - The output string holds `n` values + null markers → O(n). - Recursion stack depth = tree height: O(log n) for balanced trees, **O(n)** worst case (skewed tree). - The `deque` of tokens → O(n).

5. Alternative Approaches

A) BFS / Level-order (the reference solution) - Serialize level by level using a queue, marking nulls as `#`. - Matches LeetCode's display format exactly. - Same O(n) time/space, but uses **iterative** queue logic instead of recursion (avoids stack-overflow risk on deeply skewed trees).

B) Preorder + Inorder reconstruction - Serialize two traversals (preorder + inorder) without null markers. - ⚠️ **Only works if values are unique** — fails on duplicates. Less robust than the marker approach.

C) Compact binary encoding - For production/network use: pack values as fixed-width bytes and use a single bit per node to indicate null/non-null. - Minimizes payload size but harder to debug.

Quick comparison

Approach	Pros	Cons
DFS Preorder (shown)	Clean, elegant recursion	Stack overflow on skewed trees
BFS Level-order	Iterative, matches LeetCode	Slightly more verbose
Preorder+Inorder	No null markers	Breaks on duplicate values

Recommendation: The **DFS preorder** approach is the most elegant for interviews. If asked about robustness for very deep trees, mention switching to the **iterative BFS** version.

Codex Solution

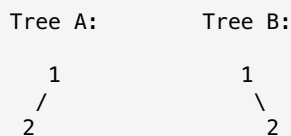
1. Intuition

The key idea is:

A binary tree can be reconstructed uniquely if we record both node values and `null` children.

If we only serialize existing node values, we lose structure information.

For example, these two trees both have preorder values `[1, 2]`:

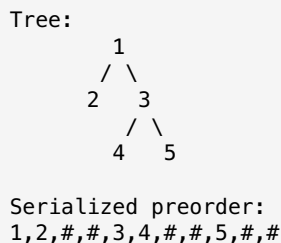


So we need a marker for missing children, such as `"#"`.

A very clean pattern is:

Use preorder DFS: `root -> left -> right`, and include `"#"` for `None`.

Example:



Because preorder always gives us the root first, we can recursively rebuild:

1. Read current token.
2. If it is `"#"`, return `None`.
3. Otherwise, create a node.

4. Recursively build its left subtree.
5. Recursively build its right subtree.

2. Approach

Serialization

Use preorder DFS.

```
serialize(root):
    tokens = []

    def dfs(node):
        if node is null:
            tokens.append("#")
            return

        tokens.append(str(node.val))
        dfs(node.left)
        dfs(node.right)

    dfs(root)

    return tokens joined by ","
```

Deserialization

Use a pointer/index to consume tokens from left to right.

```
deserialize(data):
    tokens = data split by ","
    index = 0

    def build():
        token = tokens[index]
        index += 1

        if token == "#":
            return null

        node = TreeNode(int(token))
        node.left = build()
        node.right = build()

        return node

    return build()
```

3. Python Solution

```
import sys
from typing import Optional

# Increase recursion limit because the tree can be skewed with up to 10^4 nodes.
sys.setrecursionlimit(100000)

# Definition for a binary tree node.
# LeetCode already provides this class.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Codec:
    def serialize(self, root: Optional['TreeNode']) -> str:
        ....
```

Converts a binary tree into a string.

We use preorder DFS:

node → left subtree → right subtree

For every None child, we store "#".

This is necessary to preserve the exact tree structure.
"""

tokens = []

```
def dfs(node: Optional['TreeNode']) -> None:
```

```
    # Base case: missing child
```

```
    if node is None:
```

```
        tokens.append("#")
```

```
        return
```

```
    # Store current node value
```

```
    tokens.append(str(node.val))
```

```
    # Recursively serialize left and right subtrees
```

```
    dfs(node.left)
```

```
    dfs(node.right)
```

```
dfs(root)
```

```
    # Join all tokens into one string.
```

```
    # Comma is safe because node values are integers.
```

```
    return ",".join(tokens)
```

```
def deserialize(self, data: str) -> Optional['TreeNode']:
```

```
    """
```

Converts the serialized string back into the original binary tree.

Since serialization used preorder DFS, deserialization also consumes tokens in preorder order.
"""

```
tokens = data.split(",")
```

```
    # index points to the next token to consume
```

```
    index = 0
```

```
def build() -> Optional['TreeNode']:
```

```
    nonlocal index
```

```
    # Read current token
```

```
    token = tokens[index]
```

```
    index += 1
```

```
    # "#" represents a null child
```

```
    if token == "#":
```

```
        return None
```

```
    # Otherwise, create the current node
```

```
    node = TreeNode(int(token))
```

```
    # Because preorder is root-left-right,
```

```
    # after creating the root, we build left subtree first,
```

```
    # then right subtree.
```

```
    node.left = build()
```

```
    node.right = build()
```

```
    return node
```

```
return build()
```

4. Complexity Analysis

Let n be the number of real tree nodes.

A binary tree with n nodes has exactly $n + 1$ null child pointers, so the serialized string contains:

n real node values + $n + 1$ null markers = $2n + 1$ tokens

Time Complexity

Serialization

$O(n)$

Each real node and each null child is visited once.

Deserialization

$O(n)$

Each token is consumed exactly once.

Space Complexity

Serialization

$O(n)$

We store $2n + 1$ tokens in the output list.

The recursion stack uses:

$O(h)$

where h is the height of the tree.

In the worst case, for a skewed tree:

$h = n$

So worst-case auxiliary recursion space is:

$O(n)$

Deserialization

$O(n)$

We store all tokens after splitting the string.

The recursion stack also takes $O(h)$, which is $O(n)$ in the worst case.

5. Alternative Approaches

1. BFS / Level Order Serialization

You can serialize the tree level by level using a queue.

Example:

`[1,2,3,null,null,4,5]`

This is close to LeetCode's display format.

Basic idea:

- Use a queue.
- Append node values.
- Append `"#"` for null children.
- During deserialization, use the array to assign left and right children in order.

This is iterative, so it avoids recursion depth issues.

2. Postorder DFS

You can also use postorder traversal:

```
left -> right -> root
```

But deserialization is usually done from the end of the list.

3. Preorder Without Nulls + Inorder

Another classic way to reconstruct a binary tree is using:

```
preorder + inorder
```

However, this requires node values to be unique. This problem does not guarantee uniqueness, so using null markers is simpler and more robust.

The preorder DFS with null markers is one of the cleanest and most reliable solutions for this problem.